Module 12: State Management Basics

1. Lifting State Up

📌 **Brief Notes**

- **Lifting state up** means moving state from a child component to a **common parent**.
- Used when **multiple child components need to share or update the same data**.
- Parent holds the state and passes data + functions as props.

💡 **Example Program**

```
// Parent Component
function Parent() {
  const [count, setCount] = React.useState(0);

  return (
    <>
      <ChildOne count={count} />
      <ChildTwo setCount={setCount} />
    </>
  );
}

function ChildOne({ count }) {
  return <h2>Count: {count}</h2>;
}

function ChildTwo({ setCount }) {
  return <button onClick={() => setCount(prev => prev + 1)}>Increment</button>;
```

```
}
```

🧠 **Key Point**

State is **lifted to Parent** so both children can access it.

---

2. Props Drilling

📌 **Brief Notes**

- **Props drilling** means passing data through multiple component levels.
- Can make code **hard to read and maintain**.
- Common problem in deeply nested components.

💡 **Example Program**

```
function App() {
  const user = "Alice";
  return <ComponentA user={user} />;
}

function ComponentA({ user }) {
  return <ComponentB user={user} />;
}

function ComponentB({ user }) {
  return <ComponentC user={user} />;
}

function ComponentC({ user }) {
  return <h3>Hello {user}</h3>;
}
```

⚠️ **Problem**

Component A and B don’t need user, but must pass it anyway.

---

3. Context API

📌 Brief Notes

- Context API helps **avoid props drilling**.
- Allows data to be shared **globally** within a component tree.
- Best for **themes, user info, language settings**.

💡 Example Program

```
const UserContext = React.createContext();

function App() {
  return (
    <UserContext.Provider value="Alice">
      <Component />
    </UserContext.Provider>
  );
}

function Component() {
  return <Child />;
}

function Child() {
  const user = React.useContext(UserContext);
  return <h2>Welcome {user}</h2>;
}
```

---

4. When to Use Global State

📌 Brief Notes

Use global state when:

- Many components need the same data
- Data must persist across pages
- Prop drilling becomes complex

✅ Examples of Global State

- Logged-in user
- Theme (dark/light)
- Shopping cart
- Language preferences

❌ Avoid Global State For

- Local UI state (form input, modal open/close)

Module 13: State Management with Redux

---

1. Introduction to Redux

📌 Brief Notes

- Redux is a **predictable state container** for JavaScript apps.
- Uses **single global store**.
- State updates follow **strict rules** → easier debugging.

---

2. Understanding the Need for State Management

📌 Brief Notes

Redux is useful when:

- App grows large

- Multiple components share state
- State logic becomes complex
- You need time-travel debugging

---

3. Basic Concepts: Actions, Reducers, Store

📌 Concepts Explained

Concept	Description
Action	Object describing **what happened**
Reducer	Function that **updates state**
Store	Holds the **entire app state**

---

4. Setting Up Redux

📌 Installing Redux

```
npm install redux react-redux
```

---

5. Creating Actions and Reducers

💡 Example Program

Action

```
const increment = () => {
  return { type: "INCREMENT" };
};
```

Reducer

```
const initialState = { count: 0 };

function counterReducer(state = initialState, action) {
  switch (action.type) {
```

```
    case "INCREMENT":
      return { count: state.count + 1 };
    default:
      return state;
  }
}
```

---

6. Creating the Store

```
import { createStore } from "redux";

const store = createStore(counterReducer);
export default store;
```

---

7. Connecting React with Redux

📌 **Wrap App with Provider**

```
import { Provider } from "react-redux";
import store from "./store";

<Provider store={store}>
  <App />
</Provider>
```

---

8. Using connect to Connect Components

💡 **Example Program**

```
import { connect } from "react-redux";

function Counter({ count, increment }) {
```

```
  return (
    <>
      <h2>{count}</h2>
      <button onClick={increment}>+</button>
    </>
  );
}

const mapStateToProps = (state) => ({
  count: state.count,
});

const mapDispatchToProps = (dispatch) => ({
  increment: () => dispatch({ type: "INCREMENT" }),
});

export default connect(mapStateToProps, mapDispatchToProps)(Counter);
```

---

9. Dispatching Actions

📌 **Brief Notes**

- **Dispatch** sends actions to the store
- Reducers listen and update state

```
dispatch({ type: "INCREMENT" });
```

📝 **Practice Task Questions**

Module 12 Practice Tasks

1. Create two child components that share the same counter value using **lifting state up**.
2. Build a component tree with at least **4 levels** and demonstrate **props drilling**.
3. Convert the props drilling example into a **Context API solution**.
4. Identify whether the following should use global or local state:
    - Modal visibility
    - Logged-in user
    - Theme switcher
    - Search input field

Module 13 Practice Tasks

1. Create a Redux store to manage a **counter application**.
2. Define actions for **increment and decrement**.
3. Write a reducer to handle both actions.
4. Connect a React component using connect() and display the count.
5. Add a button that dispatches an action when clicked.
6. Explain the flow: **Action** → **Reducer** → **Store** → **UI**